使用Pytorch中从头实现去噪扩散概率模型（DDPM）（附代码）

数据派THU 2024年08月09日 17:04 北京

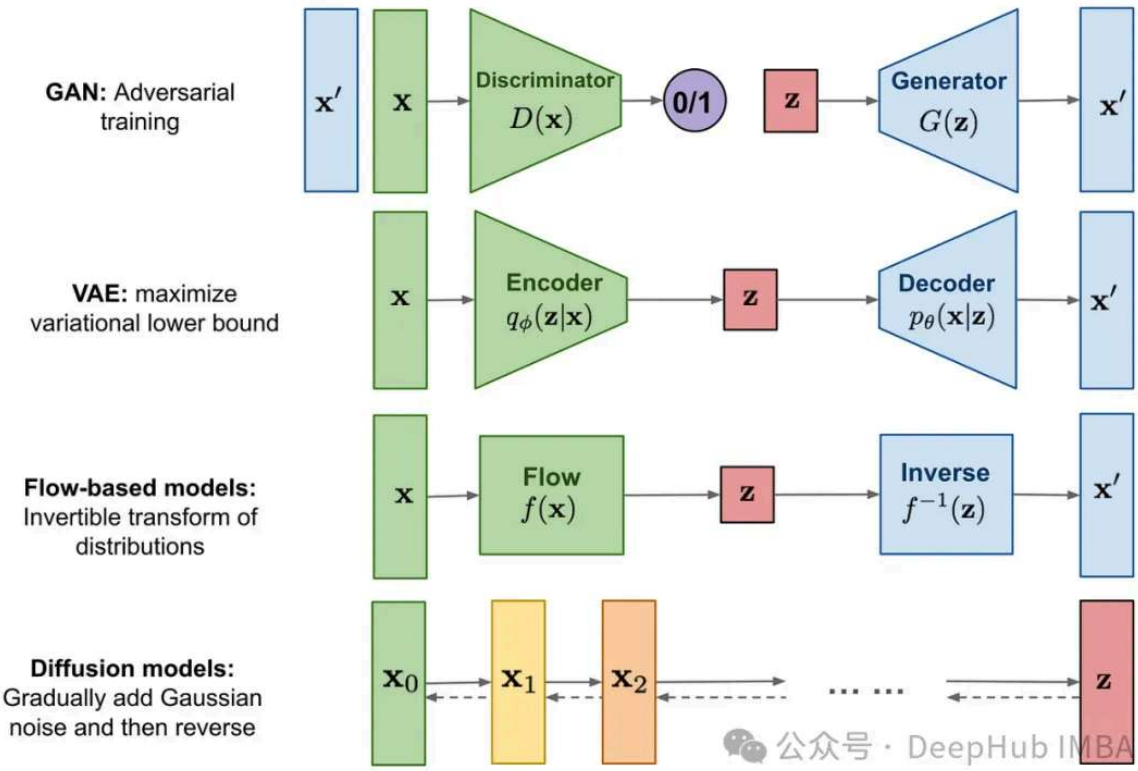
清华大数据软件团队官方微信公众号

DataPi THU, Share and Study

来源：DeepHub IMBA

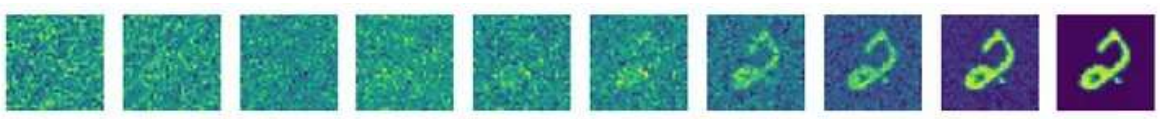
本文**约6500字**，建议阅读**13分钟**
本文将构建基础的无条件扩散模型，即去噪扩散概率模型（DDPM）。

扩散模型通常是一种生成式深度学习模型，它通过学习去噪过程来创建数据。扩散模型有许多变体，其中最流行的是条件文本模型，能够根据提示生成特定的图像。某些扩散模型（如ControlNet）甚至能将图像与某些艺术风格融合。

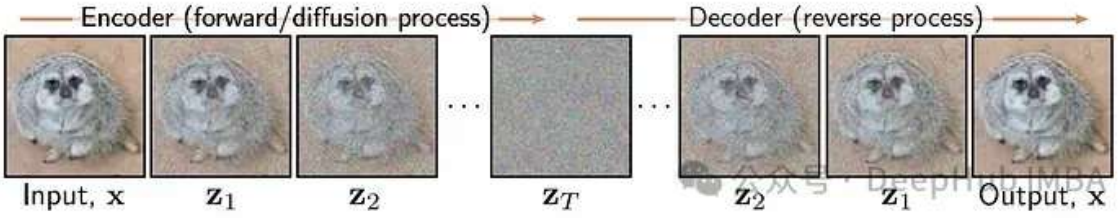


在本文中，我们将构建基础的无条件扩散模型，即去噪扩散概率模型（DDPM）。从探究算法的直观工作原理开始，然后在PyTorch中从头构建它。本文主要关注算法背后的思想和具体实现细节。

我们先展示一下本文的结果，使用扩散模型生成给MNIST的数字。



扩散模型原理



扩散过程包括正向过程和反向过程。正向过程是基于噪声计划的预定马尔可夫链。噪声计划是一组方差B1, B2, ... BT，这些方差控制组成马尔可夫链的条件正态分布。

$$q(\mathbf{z}_t|\mathbf{z}_{t-1}) = \text{Norm}_{\mathbf{z}_t}\left[\sqrt{1-\beta_t}\mathbf{z}_{t-1}, \beta_t\mathbf{I}\right] \qquad \forall\, t \in \{2, \ldots, T\}.$$

正向过程的数学表达式代表了正向过程，但直观上我们可以将其理解为一个序列，逐渐将数据示例X映射到纯噪声。在中间时间步骤t，我们得到X的带噪声版本，在最终时间步骤T，我们达到由标准正态分布大致控制的纯噪声。在构建扩散模型时，需要选择我们的噪声计划。例如，在DDPM中，噪声计划特征是从1e-4到0.02线性增加方差的1000个时间步。同样重要的是要注意正向过程是静态的，这意味着是我们选择噪声计划作为扩散模型的一个超参数，并且这个正向过程是不用训练的，因为它已经明确定义。

关于正向过程的最后一个关键细节是，因为这些分布是正态的，可以数学上推导出一个称为"扩散核"的分布，这是给定初始数据点，正向过程中任何中间值的分布。这样就可以绕过正向过程中逐步增加t-1级噪声的所有中间步骤，直接获得带有t噪声的图像，这在后期训练模型时会非常方便。这在数学上表示为：

$$q(\mathbf{z}_t|\mathbf{x}) = \text{Norm}_{\mathbf{z}_t}\left[\sqrt{\alpha_t}\cdot\mathbf{x}, (1-\alpha_t)\mathbf{I}\right]$$

这里的时间t的alpha定义为从初始时间步到当前时间步的累积（1-B）。

反向过程是扩散模型的关键。本质上是通过逐渐移除噪声从纯噪声图像生成新图像的过程。从纯噪声数据开始，对于每个时间步骤t，减去理论上由正向过程在该时间步骤添加的噪声量。通过继续移除噪声，最终得到类似于原始数据分布的东西。所以我们的主要工作是训练一个模型来近似正向过程，估计一个可以生成新样本的反向过程。

为了训练这样一个模型来估计反向扩散过程，我们可以遵循下面定义的算法：

1. 从训练数据集中随机抽取一个数据点。
2. 在噪声（方差）计划中选择一个随机时间步。
3. 添加该时间步的噪声到数据中，通过"扩散核"模拟正向扩散过程。
4. 将扩散图像传入模型，预测添加的噪声。
5. 计算预测噪声和实际噪声之间的均方误差，并通过该目标函数优化模型的参数。
6. 重复以上步骤！

通过这种方法，模型逐步学习如何有效地去除噪声，最终能够从几乎完全的噪声中恢复出与原始数据相似的图像。这种训练目标不仅帮助模型学会准确地预测噪声，而且还优化了它在各个时间步骤去噪的能力，从而使反向过程更加准确和高效。

Algorithm 18.1: Diffusion model training

Input: Training data $\mathbf{x}$
Output: Model parameters $\boldsymbol{\phi}_t$
repeat
  for $i \in \mathcal{B}$ **do** // For every training example index in batch
    $t \sim \text{Uniform}[1, \ldots T]$ // Sample random timestep
    $\boldsymbol{\epsilon} \sim \text{Norm}[\mathbf{0}, \mathbf{I}]$ // Sample noise
    $\ell_i = \left\| \mathbf{g}_t\left[\sqrt{\alpha_t}\mathbf{x}_i + \sqrt{1-\alpha_t}\boldsymbol{\epsilon}, \boldsymbol{\phi}_t\right] - \boldsymbol{\epsilon} \right\|^2$ // Compute individual loss
  Accumulate losses for batch and take gradient step
until converged

公众号 · DeepHub IMBA

在算法中，如果不看完整的推导过程的话数学公式最初看起来可能有些奇怪，但从直觉上讲，它是基于噪声计划的alpha值对扩散核进行的重新参数化，简单来说，它就是预测噪声与添加到图像中的实际噪声之间的平方差。

如果我们的模型能够成功地基于正向过程的特定时间步预测噪声量，那么就可以从时间步T的噪声开始，逐步移除每个时间步的噪声，直到恢复出与原始数据分布相似的生成样本。

采样算法可以总结如下：
1、从标准正态分布生成随机噪声。
对于从最后一个时间步开始向后移动的每个时间步：
2、通过估计反向过程分布来更新Z，该分布的均值由前一步的Z参数化，方差由该时间步模型估计的噪声参数化。
3、为了稳定性，添加少量噪声回到图像中（下面会解释原因）。
4、重复上述步骤，直到达到时间步0，这样就得到了恢复的图像！

这个过程的关键在于，每一步都精确调整噪声的移除，模拟反向扩散，从而逐步接近原始数据的分布。添加少量噪声的目的是为了防止在去噪过程中可能出现的数值不稳定性，这样可以帮助模型更平滑地逆向映射至原始数据。

Algorithm 18.2: Sampling

Input: Model, $\mathbf{g}_t[\bullet, \boldsymbol{\phi}_t]$
Output: Sample, $\mathbf{x}$
$\mathbf{z}_T \sim \text{Norm}_{\mathbf{z}}[\mathbf{0}, \mathbf{I}]$ // Sample last latent variable
for $t = T \ldots 2$ **do**
  $\hat{\mathbf{z}}_{t-1} = \frac{1}{\sqrt{1-\beta_t}}\mathbf{z}_t - \frac{\beta_t}{\sqrt{1-\alpha_t}\sqrt{1-\beta_t}}\mathbf{g}_t[\mathbf{z}_t, \boldsymbol{\phi}_t]$ // Predict previous latent variable
  $\boldsymbol{\epsilon} \sim \text{Norm}_{\boldsymbol{\epsilon}}[\mathbf{0}, \mathbf{I}]$ // Draw new noise vector
  $\mathbf{z}_{t-1} = \hat{\mathbf{z}}_{t-1} + \sigma_t\boldsymbol{\epsilon}$ // Add noise to previous latent variable
$\mathbf{x} = \frac{1}{\sqrt{1-\beta_1}}\mathbf{z}_1 - \frac{\beta_1}{\sqrt{1-\alpha_1}\sqrt{1-\beta_1}}\mathbf{g}_1[\mathbf{z}_1, \boldsymbol{\phi}_1]$ // Generate sample from $\mathbf{z}_1$ without noise

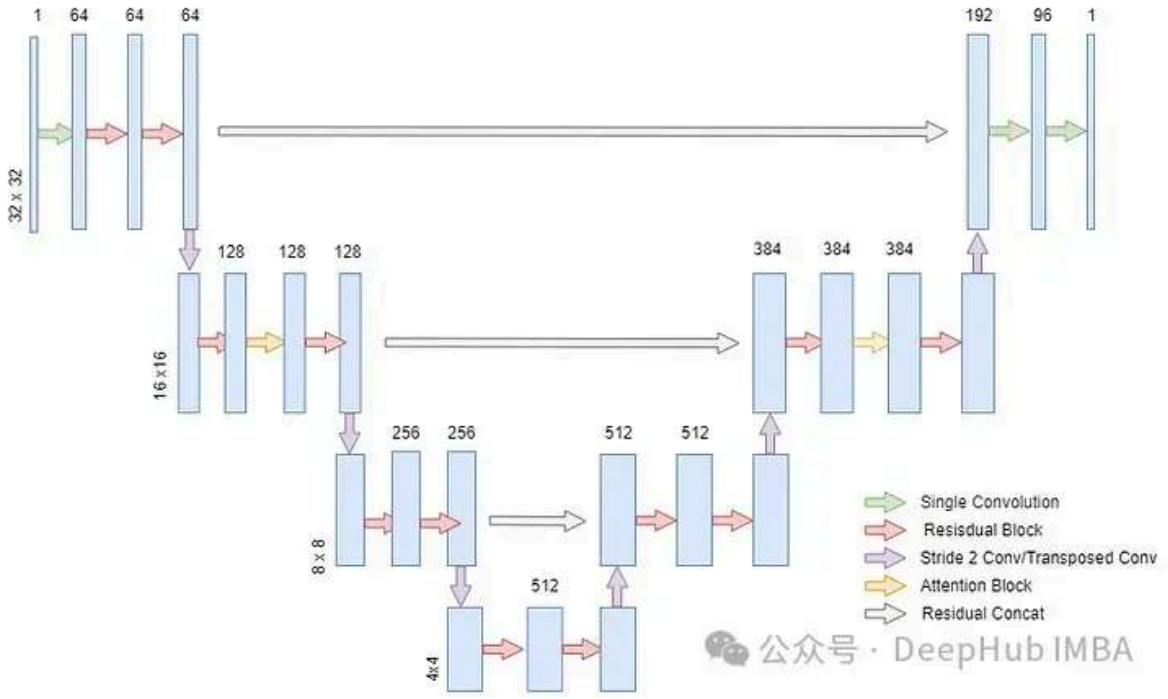
公众号 · DeepHub IMBA

生成图像的算法虽然在数学上看起来复杂，但从直觉上讲，它归结为一个迭代过程，我们从纯噪声开始，估计理论上在时间步t添加的噪声，并将其减去。我们一直这样做，直到得到我们的生成样本。需要注意的一个小细节是，在我们减去估计的噪声后，我们会加回一小部分噪声，以保持过程的稳定性。例如，一开始就一次性估计并减去全部噪声会导致非常不连贯的样本，因此在实践中，加回一点噪声并通过每个时间步迭代，已经被实证显示能生成更好的样本。

在这个过程中，核心思想是通过逐步去除每个时间步预估的噪声，然后适当地重新引入一部分噪声，这样可以避免过程中的潜在不稳定性。迭代不仅帮助恢复更精确的图像，还能确保生成过程中图像质量的连贯性和实用性。每一步的噪声重新引入虽然看似反直觉，但在实际中，这种策略对于保持整个过程的稳定性至关重要，也是实现高质量图像生成的关键步骤。

Unet

在DDPM（去噪扩散概率模型）的研究中，作者使用了最初为医学图像分割设计的UNET架构来构建模型，预测扩散反向过程中的噪声。本文中我们将使用32x32像素的图像，MNIST就是这样的数据集，但这个模型也可以扩展以处理更高分辨率的数据。UNET有许多变种，但我们将构建的模型架构概览如下图所示。



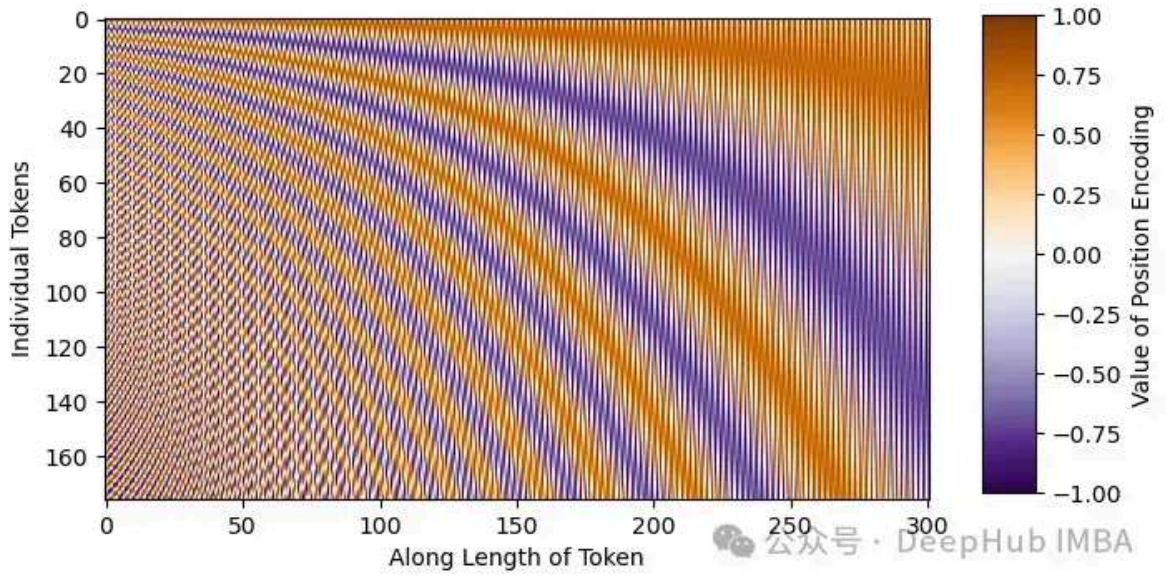
UNET是一个深度学习网络，它具有对称的编码器-解码器结构。编码器逐步降低图像的空间维度，而增加通道数，捕捉图像中的深层特征。解码器则做相反的工作，逐步恢复图像的空间维度和减少通道数，最终输出与输入图像同样大小的结果。在这个过程中，编码器和解码器之间有跳跃连接，可以帮助解码器更好地恢复图像细节。

这种架构非常适合用于图像的生成任务，因为它可以有效地处理和重建图像中的细节。在扩散模型中，UNET的任务是预测每一步中添加到图像中的噪声，这对于模型逆向去噪过程的成功至关重要。通过这种方式，UNET可以逐步减少噪声，最终恢复出清晰的图像。

DDPM UNET与经典UNET的主要区别在于DDPM UNET在16x16维度层中加入了注意力机制，并且在每个残差块中加入了正弦变换器嵌入。正弦嵌入的意义在于告诉模型我们正在尝试预测哪个时

间步的噪声。通过在噪声计划中注入位置信息，可以帮助模型预测每个时间步的噪声。例如，如果我们的噪声计划在某些时间步中有很多噪声，模型对其必须预测的时间步的理解可以帮助模型对相应时间步的噪声进行预测。

正弦嵌入是不同的正弦和余弦频率，可以直接添加到输入中，给模型额外的位置/序列理解。从下面的图像中可以看出，每个正弦波都是独一无二的，这将使模型意识到其在我们噪声计划中的位置。



模型代码实现

在我们的模型实现中，将从定义我们的导入开始并编码正弦时间步嵌入。

```
# Imports
import torch
import torch.nn as nn
import torch.nn.functional as F
from einops import rearrange #pip install einops
from typing import List
import random
import math
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
from timm.utils import ModelEmaV3 #pip install timm
from tqdm import tqdm #pip install tqdm
import matplotlib.pyplot as plt #pip install matplotlib
import torch.optim as optim
import numpy as np

class SinusoidalEmbeddings(nn.Module):
    def __init__(self, time_steps:int, embed_dim: int):
        super().__init__()
```

```
        position = torch.arange(time_steps).unsqueeze(1).float()
        div = torch.exp(torch.arange(0, embed_dim, 2).float() * -(math.log(1
        embeddings = torch.zeros(time_steps, embed_dim, requires_grad=False)
        embeddings[:, 0::2] = torch.sin(position * div)
        embeddings[:, 1::2] = torch.cos(position * div)
        self.embeddings = embeddings

    def forward(self, x, t):
        embeds = self.embeddings[t].to(x.device)
        return embeds[:, :, None, None]
```

UNET每一层中的残差块将与原始DDPM论文中使用的块使用相同的参数。每个残差块将包括以下一系列组件：

1. **组归一化（Group Norm）：**这种归一化技术是批归一化的一种变体，用于控制内部协变量偏移，特别适用于小批量大小的情况。
2. **ReLU激活函数：**这是一种非线性激活函数，它允许模型捕获输入数据中的复杂模式和非线性关系。
3. **3x3 “same” 卷积：**这种卷积操作保持输出特征图的空间尺寸与输入相同，这是通过适当的填充来实现的。
4. **Dropout：**这是一种正则化技术，通过在训练过程中随机丢弃（即设置为零）一些网络中的激活单元，来防止模型过拟合。
5. **跳跃连接（Skip Connection）：**这种连接直接将前面某层的输出传递到后面的层，这有助于解决深度网络中的梯度消失问题，并允许模型在深层中保留初级特征的信息。

```
# Residual Blocks
class ResBlock(nn.Module):
    def __init__(self, C: int, num_groups: int, dropout_prob: float):
        super().__init__()
        self.relu = nn.ReLU(inplace=True)
        self.gnorm1 = nn.GroupNorm(num_groups=num_groups, num_channels=C)
        self.gnorm2 = nn.GroupNorm(num_groups=num_groups, num_channels=C)
        self.conv1 = nn.Conv2d(C, C, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(C, C, kernel_size=3, padding=1)
        self.dropout = nn.Dropout(p=dropout_prob, inplace=True)

    def forward(self, x, embeddings):
        x = x + embeddings[:, :x.shape[1], :, :]
        r = self.conv1(self.relu(self.gnorm1(x)))
        r = self.dropout(r)
        r = self.conv2(self.relu(self.gnorm2(r)))
        return r + x
```

在DDPM中，作者在UNET的每个层（分辨率尺度）使用了两个残差块，并且在16x16维度层之间的两个残差块中加入了经典的注意力机制。下面我们将实现这种注意力机制为UNET：

```
class Attention(nn.Module):
    def __init__(self, C: int, num_heads:int , dropout_prob: float):
        super().__init__()
        self.proj1 = nn.Linear(C, C*3)
        self.proj2 = nn.Linear(C, C)
        self.num_heads = num_heads
        self.dropout_prob = dropout_prob

    def forward(self, x):
        h, w = x.shape[2:]
        x = rearrange(x, 'b c h w -> b (h w) c')
        x = self.proj1(x)
        x = rearrange(x, 'b L (C H K) -> K b H L C', K=3, H=self.num_heads)
        q,k,v = x[0], x[1], x[2]
        x = F.scaled_dot_product_attention(q,k,v, is_causal=False, dropout_p
        x = rearrange(x, 'b H (h w) C -> b h w (C H)', h=h, w=w)
        x = self.proj2(x)
        return rearrange(x, 'b h w C -> b C h w')
```

在实现注意力机制时，数据的处理相对直接。我们将数据重塑，使得高度（h）和宽度（w）的维度合并成"序列"维度，类似于传统Transformer模型的输入，而通道维度则变成嵌入特征维度。使用torch.nn.functional.scaled_dot_product_attention，因为这个实现包含了flash attention，这是一种优化版的注意力机制，从数学上与经典注意力等价。定义UNET的一个完整层：

```
class UnetLayer(nn.Module):
    def __init__(self,
                 upscale: bool,
                 attention: bool,
                 num_groups: int,
                 dropout_prob: float,
                 num_heads: int,
                 C: int):
        super().__init__()
        self.ResBlock1 = ResBlock(C=C, num_groups=num_groups, dropout_prob=d
        self.ResBlock2 = ResBlock(C=C, num_groups=num_groups, dropout_prob=d
        if upscale:
            self.conv = nn.ConvTranspose2d(C, C//2, kernel_size=4, stride=2,
        else:
            self.conv = nn.Conv2d(C, C*2, kernel_size=3, stride=2, padding=1
```

```
        if attention:
            self.attention_layer = Attention(C, num_heads=num_heads, dropout

    def forward(self, x, embeddings):
        x = self.ResBlock1(x, embeddings)
        if hasattr(self, 'attention_layer'):
            x = self.attention_layer(x)
        x = self.ResBlock2(x, embeddings)
        return self.conv(x), x
```

在DDPM中，每一层如包含两个残差块，并且可能包含一个注意力机制，此外还将嵌入传递到每个残差块中。返回的下采样或上采样的值以及之前的值，将被存储并用于残差串联的跳跃连接。

完成UNET类如下：

```
class UNET(nn.Module):
    def __init__(self,
            Channels: List = [64, 128, 256, 512, 512, 384],
            Attentions: List = [False, True, False, False, False, True],
            Upscales: List = [False, False, False, True, True, True],
            num_groups: int = 32,
            dropout_prob: float = 0.1,
            num_heads: int = 8,
            input_channels: int = 1,
            output_channels: int = 1,
            time_steps: int = 1000):
        super().__init__()
        self.num_layers = len(Channels)
        self.shallow_conv = nn.Conv2d(input_channels, Channels[0], kernel_si
        out_channels = (Channels[-1]//2)+Channels[0]
        self.late_conv = nn.Conv2d(out_channels, out_channels//2, kernel_siz
        self.output_conv = nn.Conv2d(out_channels//2, output_channels, kerne
        self.relu = nn.ReLU(inplace=True)
        self.embeddings = SinusoidalEmbeddings(time_steps=time_steps, embed_
        for i in range(self.num_layers):
            layer = UnetLayer(
                upscale=Upscales[i],
                attention=Attentions[i],
                num_groups=num_groups,
                dropout_prob=dropout_prob,
                C=Channels[i],
                num_heads=num_heads
            )
```

```
            setattr(self, f'Layer{i+1}', layer)

     def forward(self, x, t):
         x = self.shallow_conv(x)
         residuals = []
         for i in range(self.num_layers//2):
             layer = getattr(self, f'Layer{i+1}')
             embeddings = self.embeddings(x, t)
             x, r = layer(x, embeddings)
             residuals.append(r)
         for i in range(self.num_layers//2, self.num_layers):
             layer = getattr(self, f'Layer{i+1}')
             x = torch.concat((layer(x, embeddings)[0], residuals[self.num_la
         return self.output_conv(self.relu(self.late_conv(x)))
```

这个实现中与原文的唯一的区别是上游通道比UNET的典型通道稍微大一些。因为这种架构在16GB VRAM的单个GPU上训练效率更高。

调度器

为DDPM编写噪声/方差调度程序也非常简单。在DDPM中，我们的调度器将在1e-4开始，在0.02结束，并线性增加。

```
class DDPM_Scheduler(nn.Module):
    def __init__(self, num_time_steps: int=1000):
        super().__init__()
        self.beta = torch.linspace(1e-4, 0.02, num_time_steps, requires_grad=
        alpha = 1 - self.beta
        self.alpha = torch.cumprod(alpha, dim=0).requires_grad_(False)

    def forward(self, t):
        return self.beta[t], self.alpha[t]
```

同时返回beta(方差)值和alpha值，因为训练和抽样公式都是基于它们的数学推导而使用的。

最后这个函数定义了一个随机种子。如果想要重现一个特定的训练实例，每次使用相同的种子时，随机权重和优化器初始化都是相同的。

```
def set_seed(seed: int = 42):
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
```

```
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
    np.random.seed(seed)
    random.seed(seed)
```

训练代码

对于我们的实现，我们将创建一个模型来生成MNIST数据（手写数字）。由于这些图像在PyTorch中默认是28x28的，我们将图像填充到32x32，以符合原始论文中训练的32x32图像的标准。

使用Adam优化器，初始学习率设置为2e-5。我们还使用EMA（指数移动平均）来帮助提高生成质量。EMA是模型参数的加权平均，在推理时可以创建更平滑、噪声更小的样本。在这个实现中，使用了timm库的EMA V3的实现，权重设置为0.9999，与DDPM论文中所使用相同。

```
def train(batch_size: int=64,
          num_time_steps: int=1000,
          num_epochs: int=15,
          seed: int=-1,
          ema_decay: float=0.9999,
          lr=2e-5,
          checkpoint_path: str=None):
    set_seed(random.randint(0, 2**32-1)) if seed == -1 else set_seed(seed)

    train_dataset = datasets.MNIST(root='./data', train=True, download=False
    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=

    scheduler = DDPM_Scheduler(num_time_steps=num_time_steps)
    model = UNET().cuda()
    optimizer = optim.Adam(model.parameters(), lr=lr)
    ema = ModelEmaV3(model, decay=ema_decay)
    if checkpoint_path is not None:
        checkpoint = torch.load(checkpoint_path)
        model.load_state_dict(checkpoint['weights'])
        ema.load_state_dict(checkpoint['ema'])
        optimizer.load_state_dict(checkpoint['optimizer'])
    criterion = nn.MSELoss(reduction='mean')

    for i in range(num_epochs):
        total_loss = 0
        for bidx, (x,_) in enumerate(tqdm(train_loader, desc=f"Epoch {i+1}/
            x = x.cuda()
            x = F.pad(x, (2,2,2,2))
            t = torch.randint(0,num_time_steps,(batch_size,))
            e = torch.randn_like(x, requires_grad=False)
```

```
            a = scheduler.alpha[t].view(batch_size,1,1,1).cuda()
            x = (torch.sqrt(a)*x) + (torch.sqrt(1-a)*e)
            output = model(x, t)
            optimizer.zero_grad()
            loss = criterion(output, e)
            total_loss += loss.item()
            loss.backward()
            optimizer.step()
            ema.update(model)
        print(f'Epoch {i+1} | Loss {total_loss / (60000/batch_size):.5f}')

    checkpoint = {
        'weights': model.state_dict(),
        'optimizer': optimizer.state_dict(),
        'ema': ema.state_dict()
    }
    torch.save(checkpoint, 'checkpoints/ddpm_checkpoint')
```

推理

在推理阶段，只是在逆转前向过程。从纯噪声开始，现在已训练的模型可以在每个时间步预测估计的噪声，然后可以迭代生成全新的样本。从每个不同的噪声起点，可以生成一个与原始数据分布相似但独特的样本。

```
def display_reverse(images: List):
    fig, axes = plt.subplots(1, 10, figsize=(10,1))
    for i, ax in enumerate(axes.flat):
        x = images[i].squeeze(0)
        x = rearrange(x, 'c h w -> h w c')
        x = x.numpy()
        ax.imshow(x)
        ax.axis('off')
    plt.show()

def inference(checkpoint_path: str=None,
              num_time_steps: int=1000,
              ema_decay: float=0.9999, ):
    checkpoint = torch.load(checkpoint_path)
    model = UNET().cuda()
    model.load_state_dict(checkpoint['weights'])
    ema = ModelEmaV3(model, decay=ema_decay)
    ema.load_state_dict(checkpoint['ema'])
    scheduler = DDPM_Scheduler(num_time_steps=num_time_steps)
```

```
    times = [0,15,50,100,200,300,400,550,700,999]
    images = []

    with torch.no_grad():
        model = ema.module.eval()
        for i in range(10):
            z = torch.randn(1, 1, 32, 32)
            for t in reversed(range(1, num_time_steps)):
                t = [t]
                temp = (scheduler.beta[t]/( (torch.sqrt(1-scheduler.alpha[t]
                z = (1/(torch.sqrt(1-scheduler.beta[t])))*z - (temp*model(z.
                if t[0] in times:
                    images.append(z)
                e = torch.randn(1, 1, 32, 32)
                z = z + (e*torch.sqrt(scheduler.beta[t]))
            temp = scheduler.beta[0]/( (torch.sqrt(1-scheduler.alpha[0]))*(t
            x = (1/(torch.sqrt(1-scheduler.beta[0])))*z - (temp*model(z.cuda

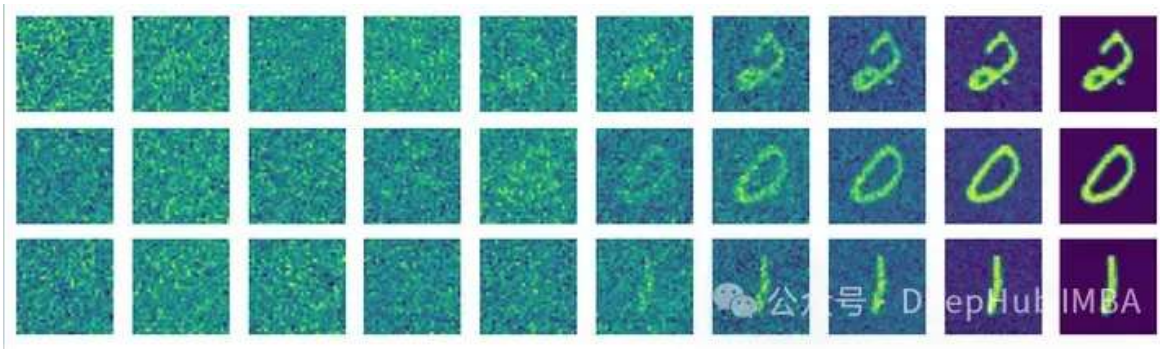
            images.append(x)
            x = rearrange(x.squeeze(0), 'c h w -> h w c').detach()
            x = x.numpy()
            plt.imshow(x)
            plt.show()
            display_reverse(images)
            images = []
```

最后如果你需要一个main函数串联训练和推理的话，就用下面这个：

```
def main():
   train(checkpoint_path='checkpoints/ddpm_checkpoint', lr=2e-5, num_epochs=7
   inference('checkpoints/ddpm_checkpoint')

if __name__ == '__main__':
   main()
```

根据上述代码进行75次训练后，得到了如下结果:



总结

以上就是我们介绍的扩散概率模型（DDPM）的实现过程。我们首先讨论了如何为生成MNIST数据创建模型，包括将图像从默认的28x28尺寸填充到32x32，以符合原论文的标准。在优化方面，我们选择了Adam优化器，并结合指数移动平均（EMA）来提高生成质量。

在模型训练部分，我们遵循了一系列明确的步骤，包括数据的噪声化、利用UNET进行预测及误差优化。我们还引入了基本的检查点机制，以便在不同的训练周期中暂停和恢复训练。推理阶段则是逆向前向过程，从纯噪声开始，通过模型逐步预测和消除噪声，最终生成与原始数据分布相似但独特的图像。

此外，我们还包括了一个辅助函数，以可视化方式展示扩散图像，帮助用户直观理解模型学习逆向过程的效果。通过这一系列的实现和优化，DDPM展现了其在图像生成和去噪方面的强大能力。

最后DDPM原始论文：
https://arxiv.org/abs/2006.11239

编辑：于腾凯
校对：梁锦程

关于我们

数据派THU作为数据科学类公众号，背靠清华大学大数据研究中心，分享前沿数据科学与大数据技术创新研究动态、持续传播数据科学知识，努力建设数据人才聚集平台、打造中国大数据最强集团军。

数据派THU
清华大数据研究中心官方平台，发布团队科研、教学等最新动态及大数据领域的相...
922篇原创内容

公众号

新浪微博：@数据派THU
微信视频号：数据派THU
今日头条：数据派THU

阅读原文